Minimum Number of Adjacent Swaps to Make Palindrome

Company: Microsoft
Round: Online Assessment
Year: 2022
Difficulty: Easy-Medium
Topic: Strings, Two Pointers, Greedy, Adjacent Swaps
Source: https://algo.monster/problems/microsoft_online_assessment_questions

Problem Description

Given a string, find the minimum number of adjacent swaps required to rearrange the string into a palindrome.An adjacent swap means swapping two characters that are next to each other.If it is impossible to rearrange the string into a palindrome, return -1.
For example : mamad can be coverted to madam in 3 swaps.

Input Format

The input contains one string `s`.
Example: mamad

Output Format

Return the minimum number of adjacent swaps needed to make the string a palindrome.
If it is not possible, return: `-1`.

Sample Input

String s = “mamad”;

Sample Output

Minimum swaps = 3

Explanation

In simple words a palindrome string is a string that reads the same from left to right and right to left or the string that remains the same when its characters are reversed.

The given string is : mamad

We want to make it : madam

Look at the first and last characters:

```
m a m a d
↑       ↑
m       d
```

For a palindrome string we need to match the first and last character. In this case they are different, so we need to find another m that can be matched with the first m.We find the second m and move it towards the end using adjacent swaps.

The m at index = 2 (3rd character) is first swapped with the a at index = 3 (4th character).

After first swap :

m a m a d

↕

m a a m d

The swapped m is now at index = 3 (4th character) and a is at index = 2 (3rd character).

Swap 2 :

Swapping the m at index = 3 (4th character) with d at index = 4 (last character).

After second swap :

m a a m d

↕

m a a d m

Now m is at index = 4 (last character) and d is at index = 3 (second last character).

First and last character matches.

As first and last character matches now we move inward to check whether the second and second last character matches.

m a a d m

↑ ↑

a d

a and d does not match.

Now we will try to find another a from the right side that is the end pointer.

m a a d m

↑

a

Move this `a` toward the `d`:

Swap 3 :

Swapping the a at index = 2 (3rd character) with d at at index = 3 (second last character).

After swap 3 :

m a a d m

↕

m a d a m

After the swaps, we get: madam

The total number of swaps required is 3.

Therefore, the answer is : 3.

Approach to Solve This Problem:

The easiest way to think about the problem is to match the characters from both ends.

We use two pointers:

- `left` starts from the beginning.
- `right` starts from the end.

Step 1: Check if a palindrome is possible

Before doing any swaps, we check the frequency of each character.

- If the string length is even, every character must occur an even number of times.
- If the string length is odd, at most one character can occur an odd number of times.

If this condition is not satisfied, we immediately return `-1`.

Step 2: Compare both ends

If:

s[left] == s[right]

then the characters are already in the correct positions.

So, we move both pointers towards the center.

Step 3: If they are different

Suppose the characters are different.

We search from the right side for a character that is equal to `s[left]`.

Once we find it, we move it towards the `right` position using adjacent swaps.

Every time we swap two neighboring characters, we increase our swap count by `1`.

Step 4: Continue until the middle

We repeat the same process until `left` crosses `right`.

The final swap count is our answer.

Java Code:

Main.java > Main

```java
import java.util.Scanner;

public class Main {

    static int minSwaps(String s) {

        char[] a = s.toCharArray();
        int n = a.length;

        // Check if palindrome is possible
        int[] freq = new int[26];

        for (char c : a) {
            freq[c - 'a']++;
        }

        int odd = 0;

        for (int x : freq) {
            if (x % 2 != 0) {
                odd++;
            }
        }

        if (odd > 1) {
            return -1;
        }

        int left = 0;
        int right = n - 1;
        int swaps = 0;

        while (left < right) {

            if (a[left] == a[right]) {
                left++;
                right--;
            }
            else {
```

```java
public class Main {
    static int minSwaps(String s) {

        while (left < right) {

            if (a[left] == a[right]) {
                left++;
                right--;
            }
            else {

                // Find matching character
                int k = right;

                while (k > left && a[k] != a[left]) {
                    k--;
                }

                // Move the character using adjacent swaps
                while (k < right) {

                    char temp = a[k];
                    a[k] = a[k + 1];
                    a[k + 1] = temp;

                    swaps++;
                    k++;
                }

                left++;
                right--;
            }
        }

        return swaps;
    }
```

```java
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print(s: "Enter a string: ");
        String s = sc.nextLine();

        int result = minSwaps(s);

        System.out.println("Minimum swaps = " + result);

        sc.close();
    }
}
```

Solution:

The code follows three simple steps:

1. Check whether a palindrome can be formed.
2. Use two pointers to compare characters from both ends.
3. If the characters do not match, find a matching character and move it using adjacent swaps.

Complexity Analysis:

Time Complexity: $O(n^2)$

In the worst case, we may have to search for a matching character and move it through several positions

Space Complexity: `O(n)`

We use a character array to perform the swaps. The frequency array has only 26 elements, so it takes constant extra space.